



Estom Paulino da Silva Junior

**Análise Comparativa de LLMs no Desenho de
Arquiteturas de Software: Modelagem de Diagramas e
Estruturas de Diretórios para Spring Boot.**

Recife
2026

Estom Paulino da Silva Junior

**Análise Comparativa de LLMs no Desenho de
Arquiteturas de Software: Modelagem de Diagramas e
Estruturas de Diretórios para Spring Boot.**

Artigo apresentado ao Curso de Bacharelado
em Sistemas de Informação da Universi-
dade Federal Rural de Pernambuco, como
requisito parcial para obtenção do título de
Bacharel em Sistemas de Informação.

Orientador: Cleyton Vanut Cordeiro de Magalhães

Universidade Federal Rural de Pernambuco – UFRPE
Departamento de Estatística e Informática
Curso de Bacharelado em Sistemas de Informação

Recife
2026

Análise Comparativa de LLMs no Desenho de Arquiteturas de Software: Modelagem de Diagramas e Estruturas de Diretórios para Spring Boot.

Estom Paulino da Silva Junior , Cleyton Vanut Cordeiro de Magalhães

¹Departamento de Estatística e Informática – Universidade Federal Rural de Pernambuco (UFRPE)
Recife – PE – Brasil

estom.paulino@ufrpe.br, cleyton.vanut@ufrpe.br

Abstract. *The adoption of Large Language Models (LLMs) in software engineering has advanced rapidly in low-level tasks such as code generation, yet their use in high-abstraction phases such as architectural design remains empirically underexplored. This paper presents a comparative empirical study evaluating the ability of four market LLMs (GPT-5.5, Claude 4.8 Opus, Gemini 3.1 Pro, and DeepSeek V4) to generate tangible architectural artifacts for the Spring Boot ecosystem. Adopting a declarative modeling paradigm (PlantUML with C4 Model notation) and a prompt-engineering strategy based on persona definition, constraint lists (allowlist/blocklist), and Chain-of-Thought reasoning, the models were tasked with designing a read-heavy URL shortener system, across three independent executions per model. Results indicate strong adherence to explicit negative constraints and convergence in trade-off decisions, with the main source of variability lying in low-level syntactic fidelity and in architectural consistency across replications, rather than in domain comprehension. We conclude that, under rigorous prompt engineering, LLMs can act as useful architectural assistants ("sounding boards"), while remaining unfit as autonomous decision-makers.*

Keywords: Software Architecture. Generative AI. LLM. Spring Boot. Prompt Engineering.

Resumo. *A adoção de Grandes Modelos de Linguagem (LLMs) na engenharia de software tem avançado rapidamente em tarefas de baixo nível, como a geração de código, mas seu uso em fases de alta abstração, como o design arquitetural, permanece pouco investigado empiricamente. Este trabalho apresenta um estudo empírico comparativo que avalia a capacidade de quatro LLMs de mercado (GPT-5.5, Claude 4.8 Opus, Gemini 3.1 Pro e DeepSeek V4) na geração de artefatos arquiteturais tangíveis para o ecossistema Spring Boot. Adotando um paradigma de modelagem declarativa (PlantUML com notação C4 Model) e uma estratégia de Engenharia de Prompt baseada em definição de Persona, listas de restrições (Allowlist/Blocklist) e Chain-of-Thought, submeteu-se aos modelos o design de um sistema Encurtador de URLs com perfil read-heavy, em três execuções independentes por modelo. Os resultados indicam alta aderência dos modelos a restrições negativas explícitas e convergência nas decisões de trade-off, com a principal fonte de variabilidade resi-*

dindo na fidelidade sintática de baixo nível e na consistência arquitetural entre replicações, e não na compreensão do domínio. Conclui-se que, sob engenharia de prompt rigorosa, LLMs podem atuar como assistentes arquiteturais ("mesas de ressonância") úteis, embora ainda inaptos como decisores autônomos.

Palavras-chave: Arquitetura de Software. Inteligência Artificial Generativa. LLM. Spring Boot. Engenharia de Prompt.

1. Introdução

A arquitetura de software representa a estrutura fundamental de um sistema, englobando seus componentes de software, as propriedades externamente visíveis desses elementos e as complexas relações entre eles (BASS; CLEMENTS; KAZMAN, 2012). Mais do que a simples escrita de código, o design arquitetural constitui uma fase crítica de tomada de decisões que afeta diretamente os atributos de qualidade de um sistema, tais como escalabilidade, manutenibilidade e segurança. Segundo Pressman (2011) e Hofmeister et al. (2007), projetar uma arquitetura exige do engenheiro um profundo conhecimento de domínio, bem como a capacidade de selecionar, adaptar e instanciar os conceitos de design e padrões mais adequados para resolver um problema específico.

Na prática, a arquitetura de *software* resolve problemas reais de modernização e escalabilidade em diferentes indústrias. No *e-commerce*, por exemplo, o uso de microsserviços divide plataformas complexas em partes menores como catálogo, pedidos e usuários, permitindo que o sistema cresça conforme a demanda e evitando que falhas pontuais derrubem o site inteiro (POZDNIAKOVA et al., 2017). No setor de seguros, empresas utilizam padrões arquiteturais para criar uma base única de *software* que pode ser rapidamente adaptada para diferentes tipos de apólices, reduzindo custos e tempo de desenvolvimento (GRUHN; STRIEMER, 2018). Além dessas aplicações tradicionais, a arquitetura também guia inovações recentes, nas quais gigantes da tecnologia, como Netflix e Uber, adaptam suas estruturas para a adoção de "*software verde*" aplicando padrões de *design* para minimizar o consumo de energia e o impacto ambiental da infraestrutura (AHUJA et al., 2024). Paralelamente, ferramentas de Inteligência Artificial já são empregadas na modernização de sistemas antigos de grandes corporações, como a Volvo (ASSUNÇÃO et al., 2025), e na organização estrutural de *softwares* para drones e carros autônomos (RUKMONO et al., 2024). Diante de toda essa complexidade exigida pelo mercado atual, torna-se urgente investigar se os LLMs conseguem, de fato, atuar como assistentes confiáveis para ajudar os engenheiros a desenhar tais arquiteturas.

Nos últimos anos, os Grandes Modelos de Linguagem (*Large Language Models* – LLMs) emergiram como ferramentas transformadoras na Engenharia de Software. Revisões sistemáticas da literatura demonstram que a adoção da Inteligência Artificial (IA) Generativa alcançou uma maturidade notável na automação de tarefas técnicas de baixo nível, como a geração de trechos de código, a detecção de vulnerabilidades e a criação de testes unitários (HOU et al., 2024). A fronteira dessa automação tem se expandido rapidamente com a introdução de agentes baseados em LLMs (*LLM-based agents*), sistemas capazes de atuar de forma autônoma e iterativa em múltiplas fases do ciclo de desenvolvimento, superando as limitações estáticas dos modelos tradicionais por meio do uso de ferramentas externas e capacidade contínua de auto-melhoria (JIN et al., 2024). Na prática, essa abordagem fundamentada em múltiplos agentes já apresenta resultados expressivos em processos como a revisão de código (*code review*); nesses cenários, IA não

apenas identifica anomalias, falhas e *code smells* que escapam a analisadores estáticos comuns, mas também atua ativamente na otimização do projeto e na educação do desenvolvedor, antecipando riscos estruturais (RASHEED et al., 2024). Contudo, apesar dessa forte evolução em fluxos de trabalho centrados no código e na sua revisão, a transição do uso dessas ferramentas para fases de alta abstração cognitiva, como o *design* de sistemas e a tomada de decisão arquitetural de larga escala, ainda é incipiente e carece de diretrizes metodológicas consolidadas (SCHMID et al., 2025).

Ao atuar de forma autônoma na modelagem de sistemas complexos, os LLMs frequentemente esbarram em limitações críticas. Estudos empíricos evidenciam que, devido à ausência de consciência contextual profunda e à dificuldade em realizar análises precisas de *trade-offs* arquiteturais, os modelos tendem a sugerir arquiteturas com acoplamento excessivo, introduzir dependências inexistentes (alucinações estruturais) e falhar na reprodutibilidade visual e estrutural em ecossistemas corporativos (FOKAEFS; KHAN; AHMADZADEH, 2025). Além disso, quando desafiados a selecionar conceitos de design e padrões arquiteturais, os modelos de linguagem frequentemente divergem do julgamento de arquitetos de software humanos, evidenciando que a IA ainda não possui a confiabilidade necessária para tomar a decisão final em cenários de incerteza (CERVANTES; CAI; KAZMAN, 2025).

Apesar destas falhas, a literatura recente aponta um caminho promissor: os LLMs não devem atuar como tomadores de decisão independentes, mas sim como poderosos assistentes e catalisadores criativos. Para que essa colaboração humano-máquina seja eficaz, o modelo atua auxiliando na geração rápida de alternativas e atua como uma “*mesa de ressonância*” (*sounding board*), cabendo ao engenheiro humano validar a cobertura de domínio e lapidar o sistema (CERVANTES; CAI; KAZMAN, 2025; FOKAEFS; KHAN; AHMADZADEH, 2025). Todavia, na indústria de desenvolvimento *backend*, há uma escassez de estudos comparativos que orientem engenheiros sobre o real desempenho destas ferramentas na geração de artefatos tangíveis.

Diante desta lacuna, a pergunta central que norteia esta pesquisa é: **Como os LLMs podem apoiar a elaboração de artefatos e decisões de arquitetura de software durante o desenvolvimento de projetos?**

Para responder a esta questão, o presente artigo tem como objetivo geral propor um conjunto de recomendações e limitações quanto ao processo de elaboração de artefatos de arquitetura suportados pelo uso de LLMs. Para atingir este propósito, definem-se os seguintes objetivos específicos:

- Avaliar a capacidade de diferentes LLMs em modelagens arquiteturais declarativas a partir de requisitos textuais;
- Analisar a eficácia dos modelos em propor estruturas de diretórios e organização de componentes coerentes para o ecossistema Spring Boot;
- Mensurar a conformidade das arquiteturas propostas pelas ferramentas frente a um conjunto explícito de restrições tecnológicas obrigatórias e proibidas.

A metodologia adotada consiste em um estudo empírico comparativo no qual um *prompt* padronizado, contendo os requisitos de um sistema corporativo, é submetido a quatro modelos de IA de mercado. A partir desta introdução, este artigo organiza-se da seguinte forma. A Seção 2 apresenta o referencial teórico e os conceitos fundamentais

que sustentam a pesquisa, abordando o ecossistema de arquitetura de software e os grandes modelos de linguagem. A Seção 3 revisa os trabalhos relacionados, contextualizando o estado da arte e o diferencial empírico deste estudo. A Seção 4 detalha a metodologia e os procedimentos operacionais adotados na estruturação dos testes, cenários e critérios de avaliação quantitativa. Em seguida, a Seção 5 apresenta os resultados obtidos e discute suas implicações técnicas e limitações estruturais. Por fim, a Seção 6 reúne as considerações finais e conclusões, destacando as principais contribuições do estudo, suas limitações e as possibilidades de trabalhos futuros.

2. Referencial Teórico

Para compreender o impacto e os desafios da inserção da Inteligência Artificial Generativa no processo de design de software, é necessário estabelecer os fundamentos da arquitetura de sistemas e o papel atual dos Grandes Modelos de Linguagem (*Large Language Models* – LLMs) no ciclo de desenvolvimento.

2.1. Arquitetura de Software, Padrões e Componentes

A arquitetura de software é a espinha dorsal de qualquer sistema computacional robusto. Bass, Clements e Kazman (2012) definem a arquitetura não apenas como a estrutura do sistema, mas como o conjunto de decisões cruciais de design que garantem o atendimento aos requisitos não funcionais (atributos de qualidade) como desempenho, segurança e disponibilidade. O processo arquitetural exige a decomposição do problema em componentes lógicos e físicos, estabelecendo regras claras de comunicação e acoplamento (PRESSMAN, 2011; HOFMEISTER et al., 2007).

Para solucionar problemas recorrentes de design, a engenharia de software consolidou o uso de Padrões de Projeto (*Design Patterns*). Segundo Gamma et al. (1995), padrões fornecem um vocabulário comum e soluções reutilizáveis para a modelagem orientada a objetos, permitindo que os sistemas sejam mais flexíveis e modulares. Em paralelo, a seleção adequada de componentes de software (COTS – *Commercial Off-The-Shelf* ou de código aberto) torna-se uma habilidade central do arquiteto, exigindo metodologias rigorosas de avaliação para garantir que os componentes integrados não degradem o desempenho ou a segurança da arquitetura global.

2.2. Grandes Modelos de Linguagem (LLMs) na Engenharia de Software

Com os avanços no Processamento de Linguagem Natural (PLN), os LLMs tornaram-se ferramentas onipresentes. Fan et al. (2023), em sua taxonomia sobre os problemas em aberto na área, destacam que as aplicações de LLMs em Engenharia de Software têm tido sucesso proeminente em atividades de implementação, como geração e complementação de código, tradução de linguagens de programação e geração automática de testes unitários.

No entanto, revisões sistemáticas da literatura (HOU et al., 2024) confirmam que o foco das ferramentas de IA continua centrado em artefatos de granularidade fina. A capacidade de abstração sistêmica, que exige o entendimento de requisitos complexos, restrições de infraestrutura e alocação de componentes em diretórios ou microsserviços, continua a ser um campo de fronteira, desafiando a confiabilidade autônoma desses modelos em tarefas de design de software.

2.3. Engenharia de *Prompt* e Aprendizagem em Contexto

A eficácia na interação com LLMs baseia-se na Engenharia de *Prompt*, que utiliza o mecanismo de aprendizagem em contexto (*in-context learning*) para guiar o comportamento do modelo sem a necessidade de retreino. Conforme sumarizado por Li (2023), o desempenho da IA varia drasticamente dependendo de como o contexto é apresentado. Dentre as principais técnicas de estruturação de *prompts*, destacam-se:

- **Zero-shot Prompting:** A tarefa é submetida ao modelo sem o fornecimento de exemplos prévios. O LLM baseia-se puramente na sua base de conhecimento intrínseca para inferir a resposta. É a abordagem ideal para avaliar a capacidade genérica da IA diante de problemas inéditos.
- **Few-shot Prompting:** O *prompt* é enriquecido com um conjunto prévio de demonstrações (exemplos de como a saída deve ser estruturada). Esta técnica melhora significativamente a precisão em tarefas que exigem adaptação a formatos rígidos (como a geração de código XML ou de árvores de diretórios).
- **Chain-of-Thought (CoT):** Consiste em induzir o modelo a expor explicitamente as etapas do seu raciocínio lógico antes de gerar a resposta final. Na arquitetura de *software*, esta técnica obriga a IA a ponderar os prós e contras (*trade-offs*) antes de sugerir a adoção de um padrão estrutural.

A seleção rigorosa destas estratégias é fundamental para mitigar alucinações estruturais e garantir a reprodutibilidade técnica na geração de artefatos de *software*.

3. Trabalhos Relacionados

A utilização de IA Generativa no design de arquiteturas de software tem atraído a atenção de pesquisadores que buscam mapear os limites cognitivos e estruturais das LLMs.

Schmid et al. (2025) realizaram uma Revisão Sistemática da Literatura cruzando o domínio de Arquitetura de Software com LLMs. Os autores evidenciaram que, embora exista um interesse crescente na utilização de IA para a recuperação de conhecimento arquitetural e documentação, ainda há uma profunda carência de abordagens empíricas que analisem a capacidade preditiva da IA na recomendação de estilos e na modelagem visual de componentes para novos sistemas.

Em um estudo empírico com foco na construção autônoma de sistemas por IA, Fokaefs, Khan e Ahmadzadeh (2025) questionaram explicitamente se a inteligência artificial é capaz de construir arquiteturas completas. Os resultados revelaram que, ao utilizar abordagens puras (*Zero-Shot*), os modelos frequentemente produzem “alucinações estruturais”. As LLMs tendem a sugerir arquiteturas com acoplamento bidirecional excessivo e falham ao tentar definir fronteiras rigorosas de domínio, evidenciando a incapacidade de manter a consistência de um projeto de alta complexidade sem supervisão estrita.

Essas limitações convergem com a tese defendida por Cervantes, Cai e Kazman (2025), que investigaram o papel das LLMs no design de arquitetura de software. Os autores concluíram que as LLMs não devem ser encaradas como tomadoras de decisão (*decision-makers*), mas sim como assistentes arquiteturais (*architectural assistants*) ou “mesas de ressonância”. O estudo demonstrou que as IAs são eficientes para propor alternativas iniciais de arquitetura (como fluxos em diagramas C4), mas que decisões finais que afetam transações, tolerância a falhas e implantação exigem o julgamento humano especializado.

3.1. Modelos de Linguagem e Agentes autônomos na Engenharia de Software

A evolução dos Grandes Modelos de Linguagem (*Large Language Models* – LLMs) redefiniu as fronteiras da automação no ciclo de vida do desenvolvimento de *software*. Inicialmente restritos a tarefas estáticas de completamento de texto, estes modelos evoluíram para sistemas dinâmicos baseados em agentes (*LLM-based agents*), nos quais a inteligência artificial atua como o núcleo central para a tomada de decisão e para a execução de ações autônomas (JIN et al., 2024). Diferente das abordagens tradicionais de interações isoladas, os agentes conseguem planejar tarefas complexas, interagir com ferramentas externas como compiladores e repositórios de código e demonstrar uma capacidade contínua de auto-melhoria através do mecanismo de aprendizagem em contexto. Esta transição expande o papel da IA de um simples gerador de trechos de código para um assistente proativo capaz de compreender e manipular a estrutura de sistemas de forma holística (JIN et al., 2024).

Um reflexo prático desta maturidade dos agentes encontra-se nos fluxos automatizados de revisão de código (*code review*). Estudos empíricos indicam que a aplicação de agentes baseados em LLM em tarefas de análise de código supera significativamente as limitações das ferramentas de análise estática convencionais (RASHEED et al., 2024). Estes modelos demonstram eficácia não apenas na identificação de *bugs* superficiais, mas também na detecção de desvios de *design* (*code smells*) e na antecipação de riscos estruturais futuros antes mesmo da integração do código no ambiente de produção. Além de mitigar o débito técnico arquitetural, essa atuação proativa apoia diretamente a aprendizagem contínua dos engenheiros de *software*, promovendo a disseminação de boas práticas de programação e o alinhamento com padrões de engenharia em projetos corporativos de larga escala (RASHEED et al., 2024).

3.2. Diferencial e Contribuição deste Trabalho

Conforme evidenciado pela Revisão Sistemática de Schmid et al. (2025), há uma nítida carência na literatura por trabalhos práticos focados na geração de artefatos arquiteturais. O presente estudo posiciona-se exatamente para preencher esta lacuna.

Enquanto os estudos empíricos recentes, como os de Fokaefs, Khan e Ahmadzadeh (2025) e Cervantes, Cai e Kazman (2025), concentram-se em avaliar as limitações teóricas da IA e a sua viabilidade abstrata como assistente, a presente pesquisa diferencia-se por adotar um enfoque estritamente pragmático e voltado à engenharia de *software backend*. Este artigo não apenas analisa se a IA compreende requisitos textuais, mas compara metodologicamente o desempenho de quatro ferramentas de mercado na geração de artefatos tangíveis. A Tabela 1 consolida a comparação entre a literatura existente e a contribuição proposta por este artigo.

Tabela 1. Comparativo entre Trabalhos Relacionados e o Presente Estudo

Trabalho	Tipo de Estudo	Foco de Análise	Entrega / Contribuição Principal
Schmid et al. (2025)	Revisão Sistemática (RSL)	Estado da arte de IA na Arquitetura	Mapeia o uso de LLMs e comprova a falta de estudos empíricos sobre modelagem visual de sistemas.
Fokaefs, Khan e Ahmadzadeh (2025)	Estudo Empírico	Arquitetura de Nuvem (Autônoma)	Avalia o modelo <i>Zero-Shot</i> e evidencia alucinações estruturais e excesso de acoplamento gerados por IA.
Cervantes, Cai e Kazman (2025)	Estudo Empírico	Tomada de Decisão Arquitetural	Conclui que a IA é ineficaz como decisora isolada, mas atua bem como assistente (“mesa de ressonância”).
Presente Estudo	Estudo Empírico Comparativo (n=3 execuções)	<i>Backend</i> Spring Boot	Compara 4 LLMs na geração de artefatos práticos (Diagramas PlantUML/C4 Model e hierarquia Spring Boot) com replicação, entregando recomendações de Engenharia de <i>Prompt</i> e um diagnóstico da estabilidade sintática entre execuções.

Fonte: Elaborado pelo autor.

Ao parametrizar a pesquisa num cenário de sistema Encurtador de URLs, que exige decisões rígidas de *cache*, banco de dados e roteamento, este estudo vai além da abstração. Entrega recomendações e restrições práticas (“o que fazer” e “o que não fazer”) sobre como utilizar a Engenharia de *Prompt* para extrair decisões arquiteturais de alta fidelidade e prontas para o desenvolvimento.

4. Metodologia

Para responder à questão central desta pesquisa “*Como os LLMs podem apoiar a elaboração de artefatos e decisões de arquitetura de software durante o desenvolvimento de projetos?*” e avaliar a capacidade das inteligências artificiais na geração de artefatos arquiteturais práticos, este estudo adota uma abordagem empírica, exploratória e comparativa. Inspirada no modelo de avaliação de *design* proposto por Cervantes, Cai e Kazman (2025), a metodologia desta pesquisa foi estruturada em cinco etapas sequenciais, adaptadas para focar na entrega de estruturas tangíveis de *software backend* em vez de decisões conceituais abstratas.

As cinco etapas metodológicas que guiam este estudo são:

Metodologia: Etapas do Estudo



Figura 1. Representação do fluxo e das etapas metodológicas do estudo.

Fonte: Elaborado pelo autor.

1. **Definição do Cenário Arquitetural Base:** Elaboração dos requisitos de um sistema de alta carga (Encurtador de URLs).
2. **Seleção de Modelos e Ferramentas:** Definição dos LLMs avaliados e das tecnologias alvo.
3. **Estruturação dos *Prompts* de Avaliação:** Construção dos comandos baseados em técnicas de Engenharia de *Prompt*.
4. **Execução do Experimento:** Submissão do cenário aos modelos e extração dos artefatos.
5. **Análise e Validação Comparativa:** Avaliação da fidelidade e precisão dos diagramas gerados e da organização de diretórios estruturada.

Ao longo deste artigo, o termo artefatos brutos designa o conjunto de saídas textuais produzidas por cada modelo em cada execução, sem qualquer edição posterior: (i) o bloco de raciocínio (*Chain-of-Thought*) do Turno 1; (ii) o código-fonte declarativo em PlantUML/C4 Model do diagrama de contêineres; (iii) a árvore de diretórios e pacotes proposta no Turno 2; e (iv) a lista de dependências de *build* (*pom.xml/build.gradle*). Esses quatro elementos, multiplicados pelas doze execuções, compõem o pacote de replicação descrito no Apêndice ??.

4.1. Etapa 1: Definição do Cenário Arquitetural Base

Para que a avaliação das LLMs reflita os desafios reais da engenharia de *software*, o cenário de teste deve simular a complexidade da indústria, muito embora este trabalho não preveja a implementação física ou a implantação do sistema em um ambiente real de produção. Sob essa premissa, foi escolhido o *design* conceitual de um sistema Encurtador de URLs. A escolha por este domínio justifica-se por sua natureza *Read-Heavy* (alta taxa de leitura em relação à escrita), oferecendo um modelo ideal para avaliar como os modelos de linguagem respondem a desafios complexos de infraestrutura e gerenciamento de carga de forma puramente estrutural e lógica.

O problema arquitetural submetido às inteligências artificiais exigirá a resolução de requisitos rigorosos, incluindo:

- **Lógica de Domínio:** A arquitetura deve prever os mecanismos de codificação das chaves (utilizando padrões como *Base62*) e expiração de acessos.
- **Isolamento Estrutural:** As IAs serão desafiadas a projetar o sistema respeitando a separação de responsabilidades (SOLID), estruturando explicitamente as camadas de *controllers*, *services*, *DTOs* e repositórios.

4.2. Etapa 2: Seleção de Modelos e Ferramentas

Para garantir a relevância da pesquisa no cenário tecnológico atual, a seleção dos LLMs não ocorreu por mera conveniência, mas foi orientada por três critérios rigorosos de amostragem, visando representatividade do estado da arte e alinhamento aos parâmetros de avaliação estabelecidos por Cervantes, Cai e Kazman (2025). Primeiro, aplicou-se o critério de fronteira e adoção: foram escolhidos os modelos mais recentes e amplamente adotados dos quatro principais provedores do mercado (OpenAI, Anthropic, Google e DeepSeek), garantindo relevância prática para o engenheiro de *software* que utiliza essas ferramentas no cotidiano. Segundo, adotou-se o critério de diversidade de filosofia e arquitetura: a amostra contempla tanto modelos proprietários (GPT-5.5, Claude 4.8 Opus, Gemini 3.1 Pro) quanto um modelo de pesos abertos (DeepSeek V4), além de abranger diferentes paradigmas: modelos focados em raciocínio lógico profundo estendido e modelos de propósito geral, o que permite observar se o desempenho arquitetural varia conforme a classe do modelo. Terceiro, exigiu-se o critério de acessibilidade pública via interface *web*, o qual é estritamente coerente com o cenário de uso empírico e real que este estudo busca simular.

Sob esses critérios, foram selecionadas as seguintes ferramentas para o experimento:

- **ChatGPT (Modelo GPT-5.5):** Escolhido por ser o modelo de fronteira da OpenAI, com amplas capacidades multimodais e vasta adoção na indústria de engenharia de *software*.

- **Claude (Modelo Claude 4.8 Opus):** Selecionado devido ao seu forte desempenho comprovado em tarefas de codificação complexa e geração de texto estruturado.
- **Gemini (Modelo Gemini 3.1 Pro):** Incluído por sua ampla capacidade de janela de contexto e processamento de dados complexos em larga escala.
- **DeepSeek (Modelo DeepSeek V4):** Escolhido por sua especialização recente e destaque em raciocínio algorítmico e resolução de tarefas lógicas profundas.

Seguindo a premissa da literatura empírica atual, a interação com estes modelos ocorrerá através das suas respectivas interfaces *web* conversacionais padrão. Esta escolha metodológica visa simular com fidelidade a forma como arquitetos e engenheiros de *software* interagem diariamente com estas ferramentas, sem a injeção de configurações de *API* ou ajustes finos de temperatura (*fine-tuning*).

Para fins de reprodutibilidade, registram-se as condições exatas de execução do experimento. As interações ocorreram entre junho e julho de 2026, exclusivamente através das interfaces web conversacionais oficiais de cada provedor, sem uso de *API* ou ajustes de parâmetros. As versões avaliadas foram GPT-5.5 (OpenAI), Claude 4.8 Opus (Anthropic) e Gemini 3.1 Pro (Google), acessadas por meio de contas pagas individuais (versões *Pro/Plus*), e o DeepSeek V4 (DeepSeek), utilizado em sua modalidade de acesso gratuito (versão Pro). Cabe registrar uma limitação inerente a esse modo de acesso: modelos disponibilizados via interface web podem sofrer atualizações silenciosas por parte dos provedores, sem alteração do nome de versão, o que significa que uma replicação futura, ainda que utilize o mesmo identificador de modelo, pode não reproduzir exatamente os mesmos artefatos aqui reportados.

Para mitigar essa limitação e responder à orientação de que o processo de replicação disponha de critério mínimo de comparação, cada um dos quatro modelos foi submetido ao protocolo completo de dois turnos em três execuções independentes (E1, E2, E3), cada uma iniciada em uma sessão de conversa nova, sem histórico ou contexto de execuções anteriores. A Tabela 2 registra as condições exatas de cada uma das doze execuções realizadas, funcionando como ficha de replicação: qualquer nova rodada do protocolo deve preencher os mesmos campos, permitindo comparação direta com os dados aqui reportados.

Tabela 2. Registro das Condições de Execução por Sessão

Modelo	Exec.	Data	Interface	Acesso Web	Histórico
GPT-5.5	E1	28/06/2026	chat.openai.com	Não	Desativado
	E2	05/07/2026			
	E3	10/07/2026			
Claude 4.8 Opus	E1	28/06/2026	claude.ai	Não	Desativado
	E2	05/07/2026			
	E3	10/07/2026			
Gemini 3.1 Pro	E1	28/06/2026	gemini.google.com	Não	Desativado
	E2	05/07/2026			
	E3	10/07/2026			
DeepSeek V4	E1	28/06/2026	chat.deepseek.com	Não	Desativado
	E2	05/07/2026			
	E3	10/07/2026			

Nota: Para todas as sessões registradas na tabela, o idioma padrão configurado foi PT-BR e uma cópia integral do histórico de geração de artefatos foi rigorosamente preservada para fins de auditoria.

Fonte: Elaborado pelo autor.

4.3. Etapa 3: Estruturação dos *Prompts* de Avaliação

Para mitigar o risco de alucinações estruturais e o acoplamento excessivo comumente gerados por LLMs em abordagens de comando único, este trabalho adota uma estratégia de Engenharia de *Prompt* Interativa e Iterativa. A interação é dividida em etapas lógicas sequenciais, permitindo que o modelo refine a sua linha de raciocínio antes de entregar os artefatos finais.

Adequação Metodológica: O Paradigma Declarativo

Considerando que Grandes Modelos de Linguagem (LLMs) são, em sua essência, motores probabilísticos de predição de *tokens* e não possuem capacidade nativa de raciocínio espacial, a exigência de diagramas em formatos posicionais estritos (como XML puro) induz ao antipadrão de “alucinação visual” (sobreposição de componentes). Para isolar a avaliação lógica da IA de suas limitações geométricas espaciais, este estudo adota o paradigma de modelagem declarativa. Os modelos foram instruídos a gerar a arquitetura através de código-fonte estruturado, delegando o cálculo topológico aos motores de renderização e focando a avaliação estritamente na precisão das escolhas arquiteturais.

O instrumento de teste foi construído utilizando a técnica de Definição de Persona, inspirada no modelo de simulação de papel especializado e definição rigorosa de restrições documentado por Cervantes, Cai e Kazman (2025). A estrutura conceitual e as diretrizes estritas do *prompt* base de inicialização (Comando 1) são apresentadas no Quadro 1.

Contexto e Persona:

Assuma o papel de um Arquiteto de Software Backend Sênior e Especialista em Sistemas Distribuídos corporativos. A sua postura é formal, técnica e analítica. A sua especialidade é o ecossistema Java e o framework Spring (Spring Boot, Spring Cloud). Você domina a Arquitetura Limpa (Clean Architecturæ), princípios SOLID e o isolamento de domínio. Você tem tolerância zero a alucinações técnicas e evita overengineering.

O Desafio:

Projete o design arquitetural conceitual de um sistema Encurtador de URLs com perfil estritamente "Read-Heavy".

Restrições Absolutas (Blocklist):

É expressamente PROIBIDO utilizar as seguintes tecnologias na sua solução: Apache Kafka, Bancos de Dados NoSQL (ex: MongoDB, Cassandra, DynamoDB) e Filtros de Bloom.

Restrições Técnicas Obrigatórias (Allowlist / RNFs):

Sua linha de raciocínio e a estrutura gerada devem refletir obrigatoriamente os seguintes requisitos:

- RNF01 (Resiliência): Retry com backoff (banco de dados).
- RNF02 (Assíncrono): Broker de mensagens (ex: SQS/RabbitMQ).
- RNF03 (Segurança): Sanitização contra open redirect/injeção.
- RNF04 (Manutenibilidade): Camadas (Controller, Service, etc).
- RNF05 (Persistência): Banco relacional (PostgreSQL/H2).
- RNF06 (Cache): Cache distribuído (Redis).
- RNF07 (Observabilidade): Logs e health check (Spring Actuator).
- RNF08 (Documentação): OpenAPI/Swagger.
- RNF09 (Configuração): 12-factor app (application.yml/env).

Diretriz de Avaliação Estrutural:

Este é um projeto conceitual. Você não deve implementar a lógica dos métodos. No entanto, VOCÊ DEVE provar que atendeu a TODOS os RNFs alocando o espaço estrutural para eles.

Tarefa - Turno 1:

Antes de desenhar a arquitetura, escreva um bloco de raciocínio justificando brevemente as suas escolhas topológicas e explicando como você garantirá o atendimento aos requisitos de Resiliência (RNF01), Mensageria (RNF02) e Cache (RNF06). Pondere os trade-offs das suas escolhas.

Após essa breve reflexão analítica, gere exclusivamente em formato de código-fonte PlantUML (utilizando as macros e a notação do C4 Model, especificamente o Diagrama de Contêineres - C4_Container), o diagrama topológico de implantação.

Você não deve tentar calcular posições X/Y. Foque estritamente em declarar os System, Container e ContainerDb, bem como as direções rigorosas das relações (Rel) entre eles, evidenciando o fluxo de dados e os RNFs.

Fonte: Elaborado pelo autor.

Quadro 1. Estrutura conceitual, restrições e diretrizes do *prompt* de inicialização (Comando 1).

A utilização deste ecossistema padronizado de comandos visa equalizar as condições iniciais do experimento e mitigar assimetrias nas instruções fornecidas aos modelos, permitindo a posterior avaliação quantitativa estrutural.

Cabe esclarecer que a abordagem adotada é *zero-shot* quanto ao fornecimento de exemplos de saída (não há demonstrações de artefatos no *prompt*) porém fortemente estruturada quanto à engenharia de comando, combinando *role-prompting*, restrições explícitas (Allowlist/Blocklist) e *Chain-of-Thought*; os eixos “ausência de exemplos” e “sofisticação do *prompt*” são, portanto, independentes. Essa supressão de exemplos foi uma decisão deliberada, visando avaliar o conhecimento latente e nativo dos modelos na interpretação de requisitos de arquitetura de *software*, sem o viés de formatações previamente injetadas. A escolha fundamenta-se em evidências empíricas recentes: Yuan et al. (2024) demonstraram que o fornecimento de exemplos (*few-shot*) não é neutro, pois as demonstrações *in-context* podem introduzir vieses que degradam o desempenho, sendo o *few-shot* frequentemente menos eficaz que o *zero-shot* em tarefas que exigem compreensão semântica profunda, uma vez que os modelos tendem a capturar correlações espúrias (*shortcut learning*) em vez de raciocinar sobre o problema. Ao suprimir os exemplos de saída, isola-se a capacidade inferencial genuína do modelo da mera replicação de um molde. Complementarmente, a mesma pesquisa evidencia que a indução de raciocínio explícito (*Chain-of-Thought*) reduz a dependência de atalhos e supera outras estratégias de *prompting*, o que fundamenta sua adoção neste estudo. Reconhece-se, contudo, que o *Chain-of-Thought* não é isento de limitações: Shaikh et al. (2023) alertam que, em domínios socialmente sensíveis, o raciocínio explícito pode amplificar vieses ressalva que, embora não se aplique diretamente ao domínio técnico aqui investigado, reforça a necessidade de validação humana dos artefatos gerados. Cabe posicionar a estratégia adotada no debate metodológico atual. Embora as técnicas individuais empregadas (Persona, restrições, *Chain-of-Thought*) pertençam ao domínio da Engenharia de *Prompt*, o desenho experimental em dois turnos com preservação do histórico configura um elemento de *context engineering* disciplina mais ampla que, segundo a literatura recente, transcende o desenho da instrução isolada para abranger a otimização sistemática de todo o *payload* de informação fornecido ao modelo (MEI et al., 2025). No presente estudo, a topologia gerada no Turno 1 passa a integrar o contexto do Turno 2 sem reinicialização da sessão, de modo que a estrutura de diretórios não é gerada isoladamente, mas condicionada às decisões arquiteturais previamente estabelecidas. Esse encadeamento deliberado de contexto entre turnos é o que assegura a coerência entre o artefato visual (diagrama) e o artefato estrutural (hierarquia de código).

4.4. Etapa 4: Execução do Experimento

A execução do experimento ocorre de forma síncrona e isolada em cada uma das quatro LLMs selecionadas, utilizando as interfaces operacionais baseadas em *web*. Para garantir a integridade comparativa, uma nova sessão de conversa (*chat*) é aberta do zero para cada execução.

O fluxo de execução interativa é conduzido estritamente em dois turnos de conversação por execução:

1. **Turno 1 - Geração Visual e Infraestrutura:** O Comando 1 é submetido. O arquivo de texto bruto contendo o código gerado pelo modelo é salvo em

sua extensão nativa (.puml) para posterior validação e compilação no motor de renderização do PlantUML.

2. **Turno 2 - Geração de Baixo Nível e Estrutura de Código:** Imediatamente após a resposta do Turno 1, sem limpar o histórico da sessão, é submetido o Comando 2 de refinamento, estruturado da seguinte forma:

Tarefa - Turno 2:

Com base no diagrama topológico em PlantUML que você acabou de propor no Turno 1, detalhe a organização técnica do projeto backend. Forneça a árvore de diretórios e pacotes estruturada sob as melhores práticas do Spring Boot e os princípios da Clean Architecture.

Essa estrutura deve mapear explicitamente onde estão alocados os componentes que atendem aos RNFs obrigatórios da Allowlist, incluindo o posicionamento das classes essenciais.

Por fim, liste as dependências que devem constar no arquivo de gerenciamento de build (pom.xml ou build.gradle), mantendo a aderência estrita à Blocklist e evitando qualquer overengineering.

Fonte: Elaborado pelo autor.

Quadro 2. Estrutura conceitual e diretrizes do *prompt* de refinamento do Turno 2 (Comando 2).

Os artefatos gerados no Turno 2 são compilados em relatórios individuais por execução para a fase subsequente de auditoria técnica. Para fins de transparência e reprodutibilidade científica, os registros integrais das interações com os modelos (*logs* conversacionais e artefatos brutos) integram o pacote de replicação deste estudo, descrito no Apêndice A.

4.5. Etapa 5: Critérios de Avaliação Quantitativa e Validação

Para garantir uma análise objetiva, mensurável e reproduzível, a avaliação dos artefatos gerados pelas LLMs afasta-se da análise puramente qualitativa e adota um modelo quantitativo de satisfação de restrições. O desempenho de cada inteligência artificial será medido através de um sistema de pontuação percentual (de 0% a 100%), onde o modelo perde pontos ao violar diretrizes arquiteturais preestabelecidas.

A avaliação baseia-se na aderência da IA a duas listas de controle:

- **Allowlist (Stack Obrigatória):** Tecnologias e conceitos exigidos pelos RNFs (ex: *Spring Boot*, *PostgreSQL/H2*, *Redis*, *RabbitMQ/SQS*, *Load Balancer*, *Clean Architecture*).
- **Blocklist (Restrições Absolutas):** Tecnologias explicitamente proibidas para forçar a IA a resolver o problema com limitações impostas (ex: proibido o uso de *Kafka* para forçar um *broker* mais simples; proibido bancos *NoSQL* como *MongoDB* para forçar a modelagem relacional do Encurtador; proibido o uso de *Filtros de Bloom*).

O critério de pontuação assume que toda LLM inicia cada execução com 100% de precisão arquitetural. A partir da auditoria da árvore de diretórios e do código declarativo PlantUML gerados, aplicam-se as seguintes penalidades cumulativas:

1. **Erro Crítico de Sintaxe (Penalidade Alta – ex: -40%):** A IA gera código declarativo (PlantUML) inválido, utilizando *tokens* inexistentes, macros não fechadas

ou sintaxe corrompida, o que impossibilita a renderização e a validação do diagrama no motor oficial.

2. **Violação de Restrição Absoluta (Penalidade Crítica – ex: -30% ou Zero):** A IA ignora a *Blocklist* e implementa uma tecnologia explicitamente proibida (ex: utilizou *Kafka* ou introduziu bancos de dados *NoSQL* como o *MongoDB*), configurando uma alucinação de instrução grave.
3. **Omissão de Componente Obrigatório (Penalidade Alta – ex: -20%):** A IA desenhou a arquitetura e a estrutura de diretórios de forma coerente, mas deixou de instanciar ou mapear um requisito vital da *Allowlist* (ex: omitiu a camada de *Cache* distribuído exigida pelo RNF06).
4. **Adição Fora do Escopo / Overengineering (Penalidade Média – ex: -10%):** A IA cumpriu os requisitos obrigatórios e respeitou as proibições, mas introduziu dependências não solicitadas e estruturalmente desnecessárias (ex: forçou injeções manuais de dependência que ferem os princípios do *Spring Boot*, ou importou bibliotecas estranhas à *stack*).

A hierarquia de pesos das penalidades não é arbitrária, mas reflete a severidade prática de cada classe de falha sob a ótica da prontidão do artefato para o desenvolvimento. A maior penalidade recai sobre o erro crítico de sintaxe, pois um diagrama que não compila inviabiliza por completo o uso do artefato, independentemente da qualidade conceitual subjacente. A violação de restrição absoluta (*Blocklist*) é penalizada de forma igualmente crítica por configurar uma alucinação de instrução que contraria diretamente um requisito explícito do projeto. A omissão de componente obrigatório recebe penalidade alta, porém inferior, por comprometer um requisito sem necessariamente invalidar o restante do artefato. Por fim, o *overengineering* recebe a menor penalidade, uma vez que representa um desvio de boas práticas que, embora indesejável, não impede a compilação nem viola requisitos funcionais ou restrições impostas.

Regra de agregação. As penalidades acima são aplicadas independentemente a cada uma das três execuções (E1, E2, E3), já que cada execução constitui uma amostra independente, gerada em sessão isolada e sem histórico compartilhado. A nota de um modelo é, portanto, a média aritmética das três notas de execução, e não um julgamento holístico único sobre o comportamento geral do modelo. Essa escolha preserva a granularidade da replicação: a Tabela 4 (Seção 5.1) reporta o resultado bruto de cada execução individualmente, enquanto a Tabela 5 reporta a média por modelo apenas como resumo comparativo – nunca como substituto da distribuição observada, que é discutida qualitativamente sempre que a média isolada seria enganosa (por exemplo, ao distinguir uma falha recorrente e idêntica de uma falha intermitente e variável entre execuções, ver Seção 5.1).

Este modelo de avaliação permite compilar os resultados numa matriz de aderência. A Tabela 3 ilustra o modelo de formatação que será utilizado na seção de Resultados para comparar o desempenho quantitativo de cada LLM testada.

Tabela 3. Modelo de Matriz de Avaliação Quantitativa das LLMs (formato ilustrativo, valores fictícios)

Modelo (LLM)	Allowlist	Blocklist	Auditoria e Penalidades	Nota
Modelo A	Atendida	Respeitada	-20% (Omissão de Componente): omitiu o <i>Load Balancer</i> exigido pela Allowlist.	80%
Modelo B	Atendida	Violada	-30% ou zero (Violação de Restrição Absoluta): utilizou banco <i>NoSQL</i> , contrariando a Blocklist.	0%
Modelo C	Atendida	Respeitada	-10% (Overengineering): inseriu <i>framework</i> externo não solicitado.	90%
Modelo D	Atendida	Respeitada	Nenhuma infração: aderência estrita a todos os RNFs e restrições.	100%

Fonte: Elaborado pelo autor.

A compilação destes dados percentuais permitirá determinar, de forma estruturada e reproduzível, qual ferramenta apresentou a maior maturidade para atuar como assistente arquitetural neste cenário, respondendo de forma exploratória ao questionamento central deste estudo.

5. Resultados e Discussão

A execução da metodologia empírica resultou na extração de artefatos arquiteturais (diagramas declarativos e árvores de diretórios) gerados por quatro modelos de fronteira: GPT-5.5, Claude 4.8 Opus, Gemini 3.1 Pro e DeepSeek V4. Esta seção apresenta a avaliação quantitativa estrutural e a subsequente análise qualitativa do raciocínio e das decisões arquiteturais tomadas pelas Inteligências Artificiais. Para fins de reprodutibilidade, os artefatos brutos e os *logs* integrais gerados no experimento encontram-se disponíveis no pacote de replicação deste estudo, apresentado no Apêndice ??.

5.1. Estabilidade Sintática entre Execuções

A replicação de três execuções por modelo revelou que a fidelidade sintática do código PlantUML gerado não é uniforme nem estática dentro de um mesmo modelo, contrariando a leitura obtida a partir de uma única execução. A Tabela 4 consolida a taxa de compilação bem-sucedida por modelo.

Tabela 4. Estabilidade Sintática entre as Três Execuções (Compilação PlantUML)

Modelo	E1	E2	E3	Taxa	Natureza da(s) falha(s)
Claude 4.8 Opus				3/3	—
Gemini 3.1 Pro				3/3	—
DeepSeek V4		×	×	1/3	E2: uso indevido de expressão não citada em argumento de macro (unquoted function/procedure cannot use expression). E3: concatenação de <i>strings</i> com operador + dentro do macro <code>Container()</code> sintaxe de linguagens de propósito geral (Java/JS) incompatível com a gramática do PlantUML/C4.
GPT-5.5	×	×	×	0/3	Falha idêntica nas três execuções, confirmada por recompilação local isolada como atribuível ao conteúdo gerado: quebra dos argumentos do macro <code>Container()</code> em múltiplas linhas, incompatível com o pré-processador do PlantUML/C4-PlantUML (ver discussão abaixo).

Fonte: Elaborado pelo autor.

Dois achados distintos emergem desta tabela e merecem tratamento diferenciado, pois não representam o mesmo tipo de falha.

DeepSeek V4 – alucinação sintática de conteúdo. Nas execuções E2 e E3, o código gerado pelo DeepSeek contém erros de sintaxe rastreáveis diretamente ao texto produzido pelo modelo. Em particular, na Execução 3, o modelo constrói os argumentos do macro `Container()` concatenando *strings* com o operador + um padrão sintático válido em linguagens de propósito geral como Java ou JavaScript, mas incompatível com a gramática de invocação de macros do PlantUML/C4-PlantUML, que exige uma única *string* literal (podendo usar `\n` interno) por argumento. Esse é um caso genuíno de contaminação sintática cross-linguagem, coerente com a literatura sobre alucinações

estruturais em geração autônoma de artefatos (FOKAEFS; KHAN; AHMADZADEH, 2025). É relevante notar que esta instabilidade contradiz a impressão inicial (obtida a partir de uma única execução) de que o DeepSeek teria sintaxe “perfeita”; a replicação revela que apenas 1 das 3 execuções de fato compilou sem erros.

GPT-5.5 – alucinação sintática de formatação, confirmada por recompilação local isolada. Uma hipótese preliminar deste estudo atribuiu a falha de compilação do GPT-5.5 à sua opção de referenciar a *stdlib* C4-PlantUML via URL bruta apontando para a branch `master` do GitHub, em vez da sintaxe de biblioteca local já que o rastro do erro reportado pelo motor corresponde a código interno da própria *stdlib*, não ao texto literal do *prompt*. Essa hipótese foi testada e refutada por recompilação dos doze artefatos brutos com o motor PlantUML oficial: das onze execuções que referenciam a *stdlib* pela mesma URL bruta (as três de Gemini, as três de Claude, as três de GPT-5.5 e as duas primeiras de DeepSeek), sete compilaram sem qualquer erro inclusive a Execução 1 do DeepSeek, que usa exatamente a mesma URL do GPT-5.5. De forma decisiva, o código do GPT-5.5 recompilado localmente, com a *stdlib* fixada em disco e sem qualquer acesso à rede, reproduziu o erro de forma idêntica ao observado nos motores online. A causa raiz é, portanto, atribuível ao próprio conteúdo gerado pelo modelo: nas três execuções, o GPT-5.5 formata consistentemente as chamadas do macro `Container()` quebrando os argumentos em múltiplas linhas, com o parêntese de fechamento em linha própria um padrão de formatação de código válido em linguagens de propósito geral, porém incompatível com o pré-processador de macros do PlantUML/C4-PlantUML. Diferentemente do DeepSeek, que apresentou dois tipos distintos de erro em execuções diferentes (uma expressão não citada em E2 e concatenação de *strings* em E3), o GPT-5.5 reproduz o mesmo erro, com a mesma formatação, nas três execuções um padrão recorrente nas três rodadas, que sugere um viés estilístico forte e consistente do modelo para esse formato de quebra de linha em chamadas de função/macro, e não uma falha estocástica pontual.

```

PlantUML version 1.2026.6 / 6287b33 [2026-06-08 16:02:56 UTC]
This version of PlantUML is 175 days old, so you should
consider upgrading from https://plantuml.com/download

[From gpt_e1.puml (line 7) ]

@startuml
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Container.puml
' convert it with additional command line argument -DRELATIVE_INCLUDE="relative/absolute" to use locally
!if %variable_exists("RELATIVE_INCLUDE")
!include ./C4_Context.puml
...
... ( skipping 13822 lines )
...
!return "0"
!$mask = $mask + $orFlags(%substr($mask1, 6, 1), %substr($mask2, 6, 1))
!if ($flag1 == "0" && $flag2 == "0")
!return "0"
!endif
!return "1"
!$mask = $mask + $orFlags(%substr($mask1, 7, 1), %substr($mask2, 7, 1))
!if ($flag1 == "0" && $flag2 == "0")
!return "0"
!return $mask
!if ($mergedMask != $mask)
%set_variable_value("$" + $elementType + "Legend", %true())
!$mask = $mergedMask
!endif
!return $stereos
!if ($link != "")
!return "[[" + $link + "]]"
!else
!return ""
Container(api,
until001

```

Figura 2. Saída do motor de renderização PlantUML para o artefato do GPT-5.5 (Execução 1) – o mesmo erro (until001) se repete de forma idêntica nas três execuções.

Fonte: Elaborado pelo autor a partir da recompilação local do artefato bruto gerado pelo GPT-5.5.

Diferentemente dos demais modelos, o GPT-5.5 não produziu, em nenhuma das três execuções, um diagrama renderizável – a Figura 2 mostra exatamente o que a ferramenta devolve ao tentar compilar o artefato bruto, e é esse o único *output* visual disponível para esse modelo neste estudo.

5.2. Desempenho Quantitativo Geral

A avaliação do desempenho das ferramentas foi consolidada na Matriz de Avaliação Quantitativa (Tabela 5), aplicando o sistema de dedução de pontos estabelecido na Seção 4, com valores médios das três execuções, aplicando a mesma penalidade de erro sintático de conteúdo a toda falha de compilação confirmada por recompilação local isolada (ver Seção 5.1). O critério primário avaliou a sintaxe, a aderência à *Allowlist* (tecnologias obrigatórias) e o rigor perante a *Blocklist* (tecnologias proibidas).

Tabela 5. Matriz Final de Avaliação Quantitativa e Satisfação de Restrições das LLMs

Modelo (LLM)	Allowlist	Blocklist	Auditoria e Penalidades	Nota Média
Claude 4.8	Atendida (3/3)	Respeitada (3/3)	Nenhuma infração em nenhuma execução. Isolamento de domínio e sintaxe consistentes nas 3 rodadas.	100%
Gemini 3.1	Atendida (3/3)	Respeitada (3/3)	-10% (Overengineering) em 1 de 3 execuções (E1): injeção manual de dependências (<code>BeansConfig.java</code>), não recorrente em E2 e E3;	96,7%
DeepSeek V4	Atendida (3/3)	Respeitada (3/3)	-40% (Erro de sintaxe de conteúdo, confirmado por recompilação local) em 2 de 3 execuções (E2, E3), de naturezas distintas entre si. Adicionalmente, inconsistência arquitetural entre execuções (monólito modular em E1 vs. multi-módulo/microserviços em E2 e E3), sem penalidade formal mas registrada como instabilidade de julgamento.	73,3%
GPT-5.5	Atendida (3/3)	Respeitada (3/3)	-40% (Erro de sintaxe de conteúdo, confirmado por recompilação local isolada) nas 3 execuções, com falha idêntica nas três execuções: formatação multi-linha do macro <code>Container()</code> , incompatível com o pré-processor PlantUML/C4-PlantUML.	60%

Notas médias recalculadas após validação da causa raiz das falhas de compilação por recompilação local isolada (Seção 5.1). Regra de agregação por execução conforme definida na Seção 4, Etapa 5.

Fonte: Elaborado pelo autor.

Os resultados indicam que sob uma técnica de *prompt* estruturada, os Grandes Modelos de Linguagem apresentam uma taxa consistente na obediência a restrições negativas explícitas (a totalidade dos modelos respeitou a *Blocklist*, abstendo-se de utilizar *Kafka* ou bancos de dados *NoSQL*). O modelo lógico de todos os LLMs também compreendeu a *Allowlist*, alocando os componentes exigidos.

No entanto, a replicação em três rodadas revela que a variação de desempenho não é um artefato de uma única amostra, mas reflete diferenças reais e nem sempre estáveis dentro de cada modelo: o verdadeiro gargalo na adoção de IAs como assistentes arquiteturais não reside na compreensão das regras de negócio ou restrições de alto nível essas se mantiveram estáveis em todas as doze execuções, mas sim na execução sintática de baixo

nível e, no caso do DeepSeek, na consistência do próprio julgamento arquitetural entre rodadas independentes.

5.3. Análise da Modelagem Declarativa (PlantUML e C4 Model)

A transição metodológica para o paradigma declarativo apresentou resultados consistentes ao eliminar o antipadrão de “alucinação espacial” em todas as doze execuções nenhum diagrama apresentou sobreposição ou conflito posicional, já que o cálculo de *layout* foi integralmente delegado ao motor de renderização. Contudo, a replicação revelou fragilidades sintáticas de naturezas distintas, já detalhadas na Seção 5.1.

Os modelos Claude 4.8 Opus e Gemini 3.1 Pro entregaram diagramas consistentemente válidos nas três execuções. O Claude destacou-se pela clareza na separação assimétrica dos fluxos de escrita (*Write API*) e leitura (*Read/Redirect API*) padrão mantido de forma estável entre E1, E2 e E3, evidenciando os protocolos de comunicação nas setas relacionais. O Gemini, por sua vez, apresentou maior variação estrutural entre execuções (ver Seção 5.4), embora sempre entregando diagramas sintaticamente válidos.

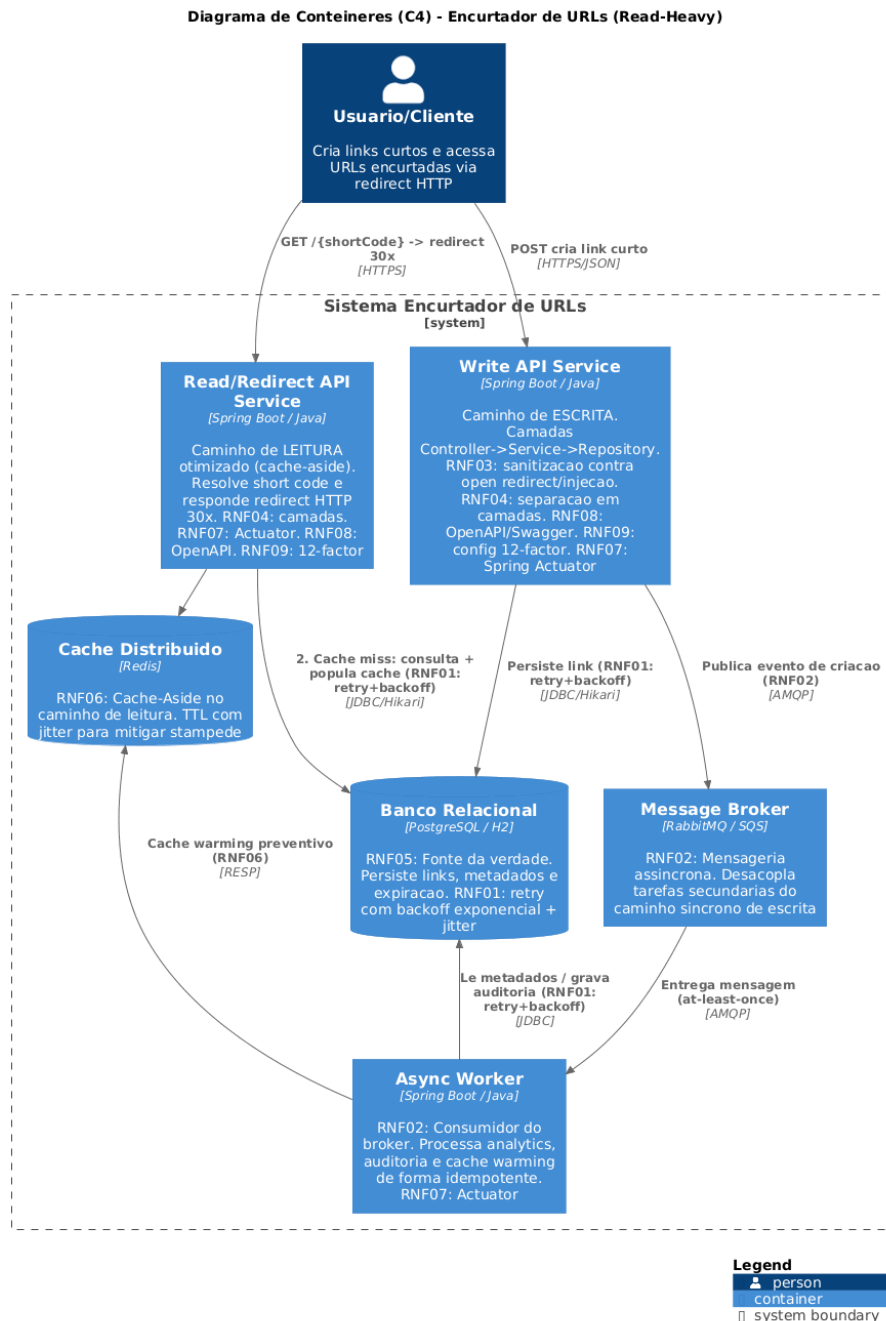


Figura 3. Diagrama de contêineres (C4 Model) gerado pelo Claude 4.8 Opus na Execução 1, renderizado a partir do artefato bruto.
 Fonte: Elaborado pelo autor a partir do artefato PlantUML gerado pelo Claude 4.8 Opus.

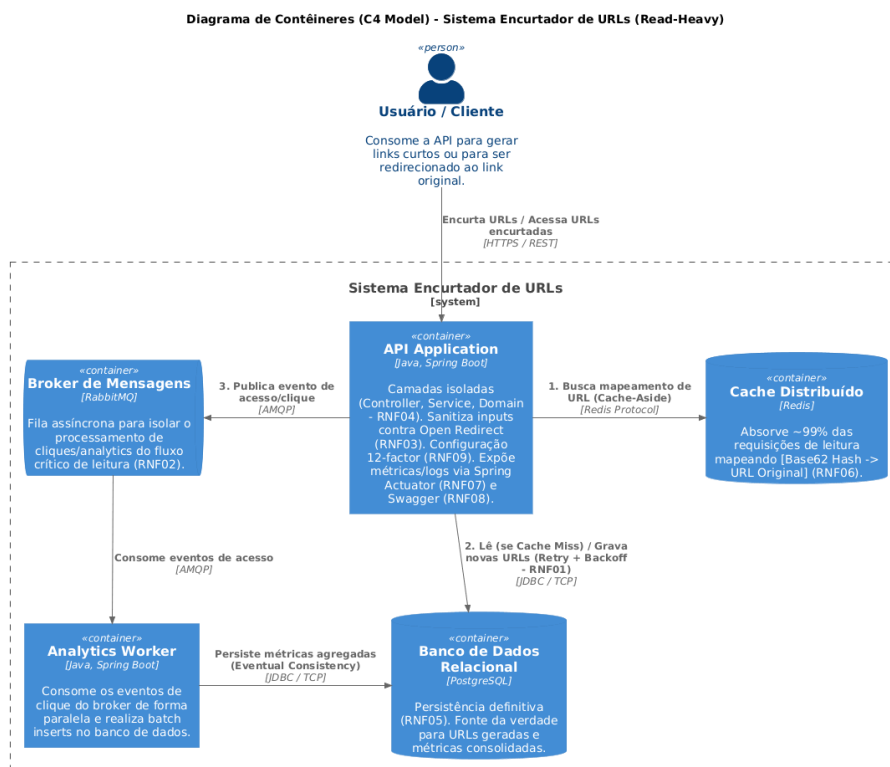


Figura 4. Diagrama de contêineres (C4 Model) gerado pelo Gemini 3.1 Pro na Execução 1, renderizado a partir do artefato bruto.

Fonte: Elaborado pelo autor a partir do artefato PlantUML gerado pelo Gemini 3.1 Pro.

"A leitura segue o padrão Cache-Aside (Look-Aside) com Redis como cache distribuído: [...] Cache hit -> redirect imediato sem tocar o PostgreSQL. Cache miss -> consulta PostgreSQL, popula o Redis com TTL e retorna. [...] Optei por não usar write-through para manter o caminho de escrita desacoplado."

Fonte: Elaborado pelo autor a partir dos artefatos gerados pelo Claude 4.8 Opus.

Quadro 3. Raciocínio do Claude 4.8 Opus sobre a estratégia de cache (RNF06) na Execução 1.

5.4. Transição para Estrutura de Código e Isolamento de Domínio

O Turno 2 do experimento auditou a capacidade das IAs de traduzirem a topologia visual para hierarquias de *software* concretas (pacotes e classes) sob o ecossistema *Spring Boot* e da *Clean Architecture*, novamente sob três execuções independentes por modelo.

Na auditoria replicada, a árvore de diretórios do **Gemini** revelou um vício arquitetural pontual em sua primeira execução: a instanciação do pacote `infrastructure/config/BeansConfig.java` com a justificativa de “injeção manual das dependências do Core” (observada explicitamente na Execução 1). No ecossistema *Spring*, cujo pilar é a Inversão de Controle (IoC) via anotações automatizadas, a configuração manual para um projeto deste escopo é considerada *overengineering* desnecessário. Nas Execuções 2 e 3, essa mesma estrutura não reapareceu, sendo substituída por configuração via anotação padrão do *framework* em ambas o que sugere que esse desvio não é determinístico, mas uma tendência ocasional do modelo.

```
infrastructure/      <-- Infraestrutura e Adaptadores
  config/            # Spring + 12-factor (RNF09)
    OpenApiConfig.java # Swagger (RNF08)
    BeansConfig.java   # Injecao manual das deps do Core

  adapters/          # Implementacao das portas
    in/web/
      UrlController.java
    out/persistence/
      UrlRepositoryAdapter.java
```

Fonte: Elaborado pelo autor a partir dos artefatos gerados pelo Gemini 3.1 Pro.

Quadro 4. Trecho da árvore de diretórios gerada pelo Gemini 3.1 Pro na Execução 1, evidenciando o `BeansConfig.java`.

O DeepSeek apresentou a maior variação estrutural entre execuções dentre os quatro modelos: na Execução 1, entregou um monólito modular único (`domain/application/infrastructure`); nas Execuções 2 e 3, sem qualquer alteração no *prompt*, migrou para uma arquitetura multi-módulo Maven com serviços fisicamente separados (`shortener-api` e `analytics-processor` na E2; um *parent* com submódulos `core`, `infrastructure`, `api` e `worker` na E3). Embora ambas as abordagens sejam defensáveis em abstrato, a divergência entre execuções idênticas do mesmo modelo é, em si, um achado relevante: indica que a granularidade de decomposição (monólito vs. microsserviços) não é uma decisão estável do DeepSeek para este cenário, o que tem implicações práticas diretas para equipes que dependam da ferramenta para gerar esqueletos de projeto reproduzíveis.

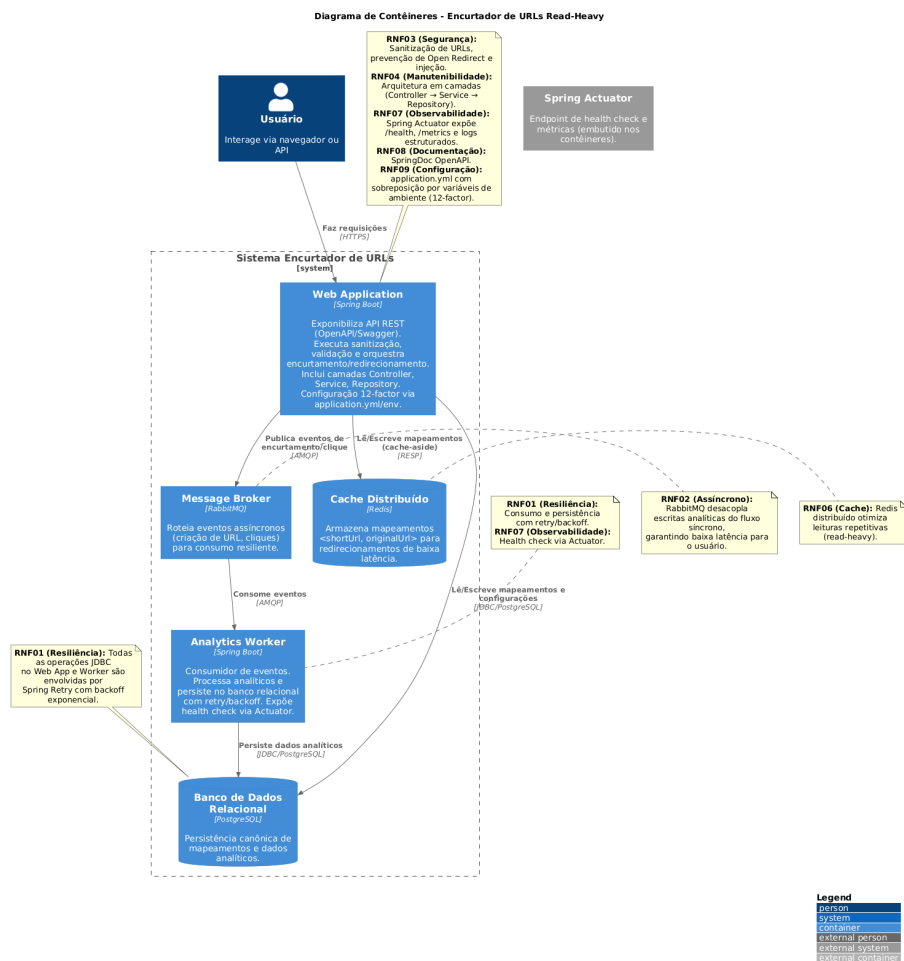


Figura 5. Diagrama de contêineres (C4 Model) gerado pelo DeepSeek V4 na Execução 1 – única das três execuções deste modelo cujo artefato compilou sem erros (ver Seção 5.1).

Fonte: Elaborado pelo autor a partir do artefato PlantUML gerado pelo DeepSeek V4.

A Figura 6 ilustra visualmente a divergência de granularidade entre a Execução 1 (monólito) e a Execução 2 (multi-módulo) do DeepSeek, a partir da raiz da árvore de diretórios gerada em cada rodada.

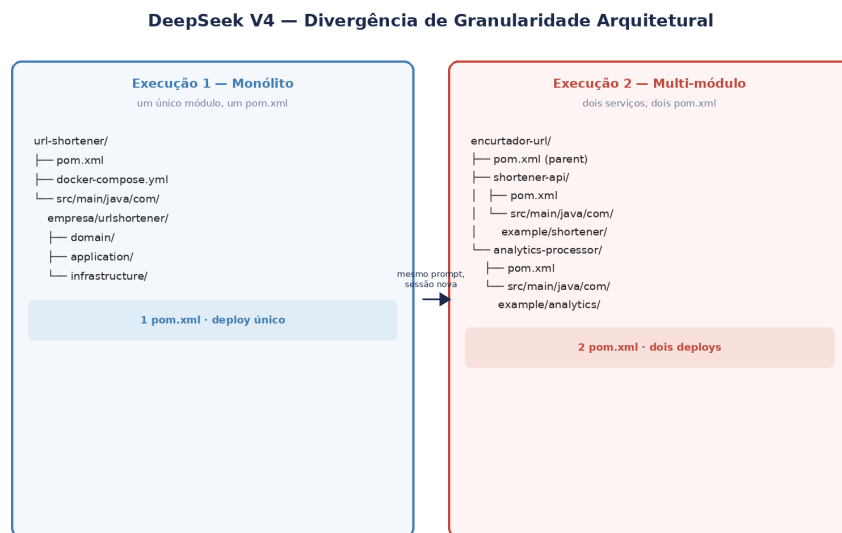


Figura 6. Contraste estrutural entre as Execuções 1 e 2 do DeepSeek V4: monólito modular vs. multi-módulo Maven, para o mesmo *prompt*.

Fonte: Elaborado pelo autor a partir dos artefatos gerados pelo DeepSeek V4.

[E1] "Com base no diagrama C4 de contêineres proposto, estruture o projeto monolito deployment-ready com separação por camadas [...]"

[E2] "O repositório é um projeto multi-módulo Maven com dois microsserviços independentes: shortener-api [...] e analytics-processor [...]"

Fonte: Elaborado pelo autor a partir dos artefatos gerados pelo DeepSeek V4.

Quadro 5. Raciocínio (*Chain-of-Thought*) do DeepSeek V4 que fundamenta a divergência estrutural entre as Execuções 1 e 2, a partir do mesmo *prompt*, em sessões independentes.

O **Claude**, em contraste, manteve em todas as três execuções o mesmo raciocínio explícito de rejeição a microsserviços/multi-módulo para este escopo, optando consistentemente por um monólito modular organizado em perfis de execução (*web-write*, *web-read*, *worker*) – com justificativa textual quase idêntica nas três rodadas sobre evitar “overengineering prematuro”.

5.5. Avaliação Cognitiva: O Fenômeno do Chain-of-Thought

A obrigatoriedade da emissão de raciocínio prévio (*Chain-of-Thought*) revelou o grau de maturidade analítica dos modelos perante *trade-offs*, de forma consistente nas doze execuções. Todas as ferramentas compreenderam, em toda e qualquer rodada, que em um sistema estritamente *Read-Heavy*, o banco de dados relacional (RNF05) constituía o principal gargalo transacional.

As respostas cognitivas revelaram um padrão de projeto homogêneo e estável entre execuções: a adoção unânime, nas doze rodadas, do padrão *Cache-Aside* com *Redis* (RNF06) na rota crítica de leitura, em conjunto com *Retry* e *Backoff* exponencial na persistência. No caminho de escrita, o raciocínio demonstrou, de forma recorrente, a urgência de utilizar o *RabbitMQ/SQS* (RNF02) em um modelo de *Fire-and-Forget* para o processamento assíncrono de *analytics*, desonerando a API síncrona.

A observação central desta etapa é que a camada de raciocínio conceitual (*Chain-of-Thought*) mostrou-se mais estável entre execuções do que a camada de execução sintática ou estrutural: mesmo nas execuções do DeepSeek e do GPT-5.5 em que houve falha de compilação ou variação arquitetural, o bloco de raciocínio textual do Turno 1 manteve-se consistente e tecnicamente correto em todas as doze rodadas. Isso reforça a leitura de que a limitação observada neste estudo está concentrada na tradução do raciocínio para artefato executável e reproduzível, não na compreensão do problema arquitetural em si.

5.6. Comparação com os Trabalhos Relacionados

Os achados deste estudo, obtidos com replicação controlada de três execuções por modelo, permitem revisitar e qualificar as conclusões da literatura apresentada na Seção 3.

Fokaefs, Khan e Ahmadzadeh (2025) reportaram que, em abordagens *Zero-Shot* para construção autônoma de arquiteturas, os LLMs frequentemente produzem alucinações estruturais e acoplamento bidirecional excessivo. Os resultados aqui obtidos qualificam essa afirmação: sob uma técnica de *prompt* estruturada (*Persona*, *Allowlist/Blocklist*, *Chain-of-Thought*), nenhuma das doze execuções apresentou acoplamento excessivo ou violação de restrição de alto nível a totalidade dos modelos respeitou a

Blocklist e alocou corretamente os componentes da Allowlist em todas as rodadas. Entretanto, a alucinação estrutural não desapareceu; ela migrou de nível: em vez de aparecer na compreensão do domínio, manifestou-se na camada sintática de baixo nível (DeepSeek, E2 e E3) e na instabilidade de decisão arquitetural entre execuções idênticas (DeepSeek, monólito *vs.* multi-módulo). Isto é, a engenharia de *prompt* rigorosa parece deslocar, e não eliminar, a fonte de alucinação relatada por Fokaefs, Khan e Ahmadzadeh (2025).

A tese central de Cervantes, Cai e Kazman (2025) de que LLMs não devem ser tratados como tomadores de decisão autônomos, mas sim como assistentes arquiteturais ou “mesas de ressonância” é consistente com os dados replicados deste estudo. A camada mais estável entre as doze execuções foi exatamente aquela em que o julgamento humano seria mais custoso de reproduzir do zero: o raciocínio conceitual sobre *trade-offs* (Seção 5.5). A camada mais instável foi exatamente aquela de menor custo cognitivo para um revisor humano corrigir: sintaxe de macro e escolha de granularidade de módulos. Isso sugere um refinamento pragmático da tese dos autores: o valor de um LLM como “mesa de ressonância” está concentrado na geração de alternativas conceituais, mas a validação humana necessária tende a recair sobre detalhes de baixo nível e não em decisões de alto nível.

Por fim, em relação à Revisão Sistemática de Schmid et al. (2025), que identificou uma lacuna de estudos empíricos sobre modelagem visual de sistemas com LLMs, o presente estudo contribui não apenas com um estudo empírico de replicação controlada (n=3 por modelo), mas com uma distinção metodológica que a literatura consultada não aborda explicitamente: a robustez de um achado de falha sintática em artefatos declarativos gerados por LLM depende de isolar, por recompilação controlada e sem acesso à rede, se a causa é o conteúdo textual do modelo ou um fator ambiental (versão de biblioteca, motor de renderização, dependência externa) como ilustrado pelo próprio caso do GPT-5.5 (Seção 5.1), cuja causa real só pôde ser estabelecida após a refutação de uma hipótese ambiental inicialmente plausível. Essa distinção entre alucinação sintática determinística (GPT-5.5) e não determinística (DeepSeek, cujas falhas variam de natureza entre execuções), e a exigência metodológica de validá-la por recompilação isolada antes de atribuir causa, são refinamentos que este estudo recomenda a trabalhos futuros que avaliem artefatos declarativos gerados por LLM.

5.7. Ameaças à Validade

A presente pesquisa empírica está sujeita a limitações que circunscrevem o alcance de suas conclusões. Seguindo a recomendação metodológica de separar as ameaças por natureza, esta seção é organizada em quatro subseções: validade interna, validade externa, validade de construção e confiabilidade.

5.7.1. Validade Interna

A validade interna deste estudo pode ser influenciada por três fatores principais. Primeiro, o *prompt* utilizado, embora padronizado entre modelos, foi desenhado por um único pesquisador; escolhas de fraseamento, ordem dos RNFs e nível de detalhe da Persona podem ter favorecido determinados modelos de forma não intencional. Segundo, a auditoria dos artefatos (classificação de penalidades, julgamento sobre o que constitui *overengineering*)

foi conduzida por um avaliador único, sem verificação cega por um segundo avaliador independente, o que introduz risco de viés de confirmação na aplicação da rubrica risco parcialmente mitigado, mas não eliminado, pela ampliação para três execuções por modelo neste estudo (anteriormente uma única execução por modelo). Terceiro, mesmo com a replicação em três rodadas, o tamanho amostral ($n=3$) permanece pequeno para inferências estatísticas robustas; os resultados devem ser lidos como ilustrativos de uma tendência observada, não como medidas com significância estatística formal.

5.7.2. Validade Externa

A generalização dos achados é limitada pelo escopo estreito do experimento: um único cenário arquitetural (Encurtador de URLs *Read-Heavy*), um único *framework* alvo (Spring Boot/ecossistema Java) e apenas quatro modelos de linguagem, todos de fronteira comercial no momento da coleta (28/06 a 10/07/2026). Não é possível afirmar que os padrões de estabilidade sintática e de consistência arquitetural aqui observados se generalizem para outros domínios de sistema (ex.: processamento de eventos *write-heavy*, sistemas de *machine learning pipelines*), para outras *stacks* tecnológicas (ex.: Node.js, .NET, Go) ou para outros LLMs e versões futuras dos mesmos modelos, dada a natureza de atualização contínua e não versionada das interfaces web utilizadas.

5.7.3. Validade de Construção

A rubrica quantitativa adotada (Seção 4) mede primariamente conformidade aderência a Allowlist, Blocklist e ausência de erros sintáticos e não captura integralmente a qualidade arquitetural de uma solução. Conforme observado nos resultados (Seção 5), duas propostas podem obter a mesma pontuação de conformidade (ex.: Claude e Gemini, quando ambos compilam e respeitam as listas de controle) e ainda assim divergirem substancialmente em atributos de qualidade não mensurados pela rubrica, como legibilidade do código gerado, coesão dos nomes de pacotes, ou adequação da granularidade de módulos ao contexto de negócio. A rubrica também não pondera a gravidade relativa de diferentes tipos de *overengineering*, tratando-os com uma penalidade fixa independentemente do impacto real na manutenibilidade do sistema.

5.7.4. Confiabilidade

Os modelos avaliados são acessados via interface web comercial, sujeita a atualizações silenciosas do provedor sem alteração do identificador de versão exposto ao usuário; a temperatura de geração não é controlada pelo pesquisador nessas interfaces, o que introduz uma fonte de variância estocástica não isolável das demais fontes de variação observadas. O próprio processo de análise deste estudo ilustra um risco adicional de confiabilidade metodológica: uma hipótese preliminar sobre a causa da falha de compilação do GPT-5.5 foi refutada por recompilação controlada e isolada de rede (Seção 5.1), revelando uma causa distinta da inicialmente suposta. Esse episódio reforça que conclusões sobre a causa raiz de falhas de compilação em artefatos gerados por LLM devem ser validadas por recompilação isolada e controlada sem acesso à rede antes de serem reportadas,

e não inferidas apenas a partir do rastro textual do erro; trata-se de uma recomendação metodológica que este estudo incorpora retroativamente e recomenda para replicações futuras. Registra-se ainda que o acesso à web permaneceu **desativado** nas doze sessões, o que fortalece a validade interna dos achados sobre estabilidade sintática relatados na Seção 5.1: com essa variável controlada, descarta-se a hipótese alternativa de que a instabilidade observada em especial no DeepSeek, cujas falhas variam de natureza entre execuções decorra de documentação externa consultada pelo modelo durante a geração, e não do próprio comportamento do modelo. Essa condição de controle é particularmente relevante para o achado do GPT-5.5: a falha idêntica e recorrente nas três execuções, já sem acesso à web como variável concorrente, sustenta com mais força a interpretação de que se trata de um viés estilístico intrínseco do modelo, e não de um artefato de busca externa.

6. Conclusão

Este estudo teve como objetivo central investigar como os Grandes Modelos de Linguagem (LLMs) podem apoiar a elaboração de artefatos e decisões de arquitetura de *software* durante o desenvolvimento de projetos. Por meio de um experimento empírico de caráter exploratório, baseado no paradigma declarativo, em diretrizes estritas de Engenharia de *Prompt* e em replicação controlada de três execuções por modelo, foi possível reunir indícios que endereçam os objetivos específicos propostos. Cabe reiterar, contudo, que o escopo do estudo (um único cenário arquitetural, um único *framework* alvo e três execuções por modelo) circunscreve o alcance das conclusões: os achados devem ser lidos como tendências observadas nas condições descritas, e não como medidas com significância estatística ou generalizáveis a outros contextos.

Em resposta ao **primeiro objetivo específico**, que visava avaliar a capacidade de geração de modelagens arquiteturais declarativas, os resultados sugerem que os modelos avaliados são capazes de traduzir requisitos textuais em topologias coerentes, mas permanecem suscetíveis a falhas sintáticas de baixo nível cuja origem exige investigação cuidadosa antes de ser atribuída ao modelo. O uso de abordagens declarativas (como *PlantUML* e notação *C4 Model*) evitou a “alucinação espacial” nas doze execuções auditadas, ao delegar a renderização visual à ferramenta adequada. A replicação indicou, porém, que a instabilidade sintática observada em todos os casos é atribuível ao conteúdo gerado pelo modelo, conforme verificado por recompilação local isolada, sem acesso à rede manifestou-se de formas distintas: como um padrão de formatação reproduzido de forma idêntica nas três execuções (GPT-5.5, quebra dos argumentos do macro `Container()` em múltiplas linhas), e como erros sintáticos de natureza variável entre execuções (DeepSeek, com dois tipos distintos de erro em E2 e E3). Trata-se de uma distinção que este estudo sugere que trabalhos futuros considerem em suas rubricas, sempre validada por recompilação controlada antes de se atribuir causa.

No que tange ao **segundo objetivo específico**, focado na análise da eficácia em propor estruturas de diretórios coerentes com o ecossistema *Spring Boot* e o isolamento de domínio, os resultados replicados apontam um quadro mais nuançado do que uma execução única sugeriria. Enquanto o Claude manteve a mesma decisão arquitetural nas três execuções (monólito modular com perfis de execução), o DeepSeek alternou entre monólito e multi-módulo/microserviços em rodadas conduzidas sob condições idênticas, o que sugere que a granularidade de decomposição pode não ser, para esse modelo e neste

cenário, uma decisão estável. Registrou-se ainda um caso de possível *overengineering* no Gemini (injeção manual de dependências), observado em uma das execuções e sem recorrência nas demais rodadas auditadas ressalva importante, uma vez que um único episódio não permite caracterizar o comportamento como sistemático.

Por fim, respondendo ao **terceiro objetivo específico**, a mensuração quantitativa replicada revelou um dado favorável e um ponto de atenção. Por um lado, as doze execuções respeitaram as restrições negativas explícitas (a Blocklist), abstendo-se de tecnologias proibidas como Kafka ou bancos de dados NoSQL em todas as rodadas. Por outro, a pontuação de conformidade estrutural variou entre 60% e 100% entre os modelos, sendo essa variação atribuível sobretudo a falhas de compilação recorrentes nas três execuções, no caso do GPT-5.5, e confirmadas por recompilação local isolada e não a lacunas de compreensão do domínio o que é consistente com a interpretação de que as limitações observadas concentram-se em detalhes de implementação e, no caso do DeepSeek, na consistência do próprio julgamento arquitetural entre execuções repetidas. Em linha com o escopo definido para este objetivo, os percentuais reportados medem conformidade a um conjunto explícito de restrições não a qualidade arquitetural das soluções em sentido amplo, dimensão cuja avaliação demandaria julgamento humano especializado (ver Seção 5.7).

De forma geral, este trabalho confirma a literatura recente ao concluir que os LLMs não estão prontos para atuar como tomadores de decisão arquitetural autônomos em um ambiente corporativo real. Os resultados aqui obtidos são consistentes com essa leitura: ainda que balizados por uma Engenharia de *Prompt* metódica combinando a definição de *Persona*, a imposição de *Allowlists* e *Blocklists* e a exigência de reflexão analítica (*Chain-of-Thought*) prévia, os modelos avaliados exigiram supervisão humana em todas as execuções que apresentaram falha. Nas condições deste estudo, tais ferramentas mostraram-se úteis como assistentes arquiteturais (“mesas de ressonância”), auxiliando na proposição de alternativas iniciais e na alocação estrutural dos Requisitos Não Funcionais especificados. Um indício particularmente relevante, embora obtido em escopo limitado e que merece confirmação em estudos mais amplos, é que o esforço de validação humana pode concentrar-se menos na revisão conceitual de alto nível camada que se mostrou estável entre as doze execuções e mais na verificação sintática e estrutural de baixo nível, onde se concentraram as falhas observadas.

Em atendimento ao objetivo geral desta pesquisa propor um conjunto de recomendações e limitações quanto ao processo de elaboração de artefatos de arquitetura suportados por LLMs, as principais recomendações práticas decorrentes deste estudo são: (i) validar toda falha de compilação de artefatos declarativos por recompilação isolada de rede antes de atribuir sua causa ao modelo ou a um fator ambiental; (ii) tratar a granularidade arquitetural (monólito vs. microsserviços) proposta por um LLM como uma decisão que exige revisão humana explícita, e não como uma saída padronizável ou confiável da ferramenta; e (iii) concentrar o esforço de revisão humana prioritariamente na camada sintática e estrutural de baixo nível dos artefatos gerados, e não apenas na revisão conceitual de alto nível, dado que esta última se mostrou consideravelmente mais estável nas condições deste estudo. As limitações do processo que sustentam essas recomendações amostra reduzida, avaliador único e escopo de cenário e *stack* restrito estão detalhadas na Seção 5.7.

6.1. Trabalhos Futuros

O presente estudo abre caminhos para explorações adicionais na interseção entre Inteligência Artificial Generativa e Engenharia de Software. Com base nas limitações observadas e no potencial destas ferramentas, delineiam-se três direções principais para investigações futuras:

- **Ampliação do Tamanho Amostral:** Este estudo avançou de uma execução única ($n=1$) para três execuções por modelo ($n=3$), o que já permitiu identificar instabilidades antes invisíveis. Trabalhos futuros devem ampliar ainda mais essa amostra (por exemplo, $n=10$ ou $n=20$ por modelo), reportando taxas de sucesso com intervalos de confiança e permitindo testes estatísticos formais de diferença entre modelos.
- **Agentes Cooperativos de *Software Architecture*:** Evoluir a interação de comandos únicos para sistemas multi-agente (*LLM-based agents*). Neste cenário, um agente atuaria como "Arquiteto" (geração de topologia), enquanto um segundo agente atuaria como "Auditor" (validação de regras e compilação sintática), criando um ciclo de *feedback* autônomo capaz de corrigir alucinações estruturais antes da entrega ao desenvolvedor humano.
- **Robustez a Variações de Formatação em DSLs Declarativas:** Investigar sistematicamente se diferentes LLMs apresentam vieses estilísticos recorrentes ao gerar linguagens declarativas (quebra de linha, indentação, concatenação de strings) e se instruções explícitas de formatação no *prompt* mitigam esse tipo de falha.

Agradecimentos

Gostaria de expressar minha profunda gratidão ao meu orientador, Cleyton Vanut, pela paciência e direcionamento. Agradeço também à minha família, ao meu amor e aos amigos por todo o suporte emocional durante esta jornada, e a todos que contribuíram direta ou indiretamente para a conclusão deste trabalho.

Referências

- AHUJA, M. et al. Automatically Assessing Software Architecture Compliance With Green Software Patterns. In: **IEEE International Conference on Software Architecture (ICSA) / GREEN**. IEEE, 2024.
- ASSUNÇÃO, W. et al. Experiences on Using Large Language Models to Re-Engineer a Legacy System at Volvo Group. In: **IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)**. IEEE, 2025.
- BASS, L.; CLEMENTS, P.; KAZMAN, R. **Software Architecture in Practice**. 3. ed. Boston: Addison-Wesley Professional, 2012.
- CERVANTES, H.; CAI, Y.; KAZMAN, R. **LLMs as Assistants in Software Architecture Design**. *IEEE Software*, 2025.
- FAN, A. et al. **Large Language Models for Software Engineering: Survey and Open Problems**. *IEEE*, 2023.
- FOKAEFS, M.; KHAN, H.; AHMADZADEH, B. **Can AI Build Systems? An Exploratory Study on Generating Software Architecture with LLMs**. In: *CASCON*, 2025.

- GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. **Design Patterns: Elements of Reusable Object-Oriented Software**. Boston: Addison-Wesley, 1995.
- GRUHN, V.; STRIEMER, R. (Eds.). **The Essence of Software Engineering**. Springer, 2018.
- HOFMEISTER, C. et al. **Generalizing a model of software architecture design from five industrial approaches**. *Journal of Systems and Software*, v. 80, n. 4, p. 520-531, 2007.
- HOU, X. et al. **Large Language Models for Software Engineering: A Systematic Literature Review**. *ACM Transactions on Software Engineering and Methodology*, ACM, 2024.
- JIN, H. et al. **From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future**. *arXiv preprint*, 2024.
- LI, Y. A Practical Survey on Zero-shot Prompt Design for In-context Learning. In: **Proceedings of Recent Advances in Natural Language Processing (RANLP)**. Varna, Bulgária, p. 641–647, 2023.
- POZDNIAKOVA, K. et al. Systematic Literature Review of the Cloud-ready Software Architecture. **Baltic Journal of Modern Computing**, v. 5, n. 1, p. 123-136, 2017.
- PRESSMAN, R. S. **Engenharia de Software: Uma Abordagem Profissional**. 7. ed. Porto Alegre: AMGH Editora, 2011.
- RASHEED, Z. et al. **AI-powered Code Review with LLMs: Early Results**. *arXiv preprint*, 2024.
- RUKMONO, S. A. et al. **Towards Automated Software Architecture Explanation**. 2024.
- SCHMID, L. et al. **Software Architecture Meets LLMs: A Systematic Literature Review**. *arXiv preprint* arXiv:2505.16697, 2025.
- MEI, L. et al. **A Survey of Context Engineering for Large Language Models**. *arXiv preprint* arXiv:2507.13334, 2025.
- YUAN, Y. et al. **Do LLMs Overcome Shortcut Learning? An Evaluation of Shortcut Challenges in Large Language Models**. *arXiv preprint* arXiv:2410.13343, 2024.
- SHAIKH, O. et al. **On Second Thought, Let's Not Think Step by Step! Bias and Toxicity in Zero-Shot Reasoning**. *arXiv preprint* arXiv:2212.08061, 2023.

A. Pacote de Replicação

A totalidade dos artefatos brutos e visuais gerados na execução dos testes operacionais (compreendendo os arquivos de código-fonte declarativo PlantUML, os logs completos de diálogo conversacional, as representações das hierarquias de código do Spring Boot e os diagramas gráficos de contêineres renderizados) foi integralmente consolidada e disponibilizada em repositório público para fins de auditoria acadêmica, reprodutibilidade científica e verificação de integridade metodológica. O acervo de dados pode ser acessado publicamente por meio do endereço eletrônico:

<https://github.com/EstomJr/tcc-llm-architecture>.